



Data Structure

(Course Code: 25B11CI318)

Binary Search Trees

Mr. Sandeep Kumar Patel

Assistant Professor

Jaypee University of Information Technology

Waknaghat, Solan, HP-173234

What is a Binary Search Tree?

Definition

A binary search tree is a binary tree in which, for every node x :

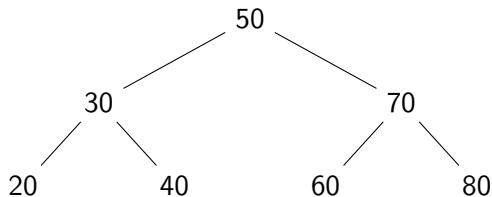
- All keys in its left subtree are smaller than $x.key$.
- All keys in its right subtree are larger than $x.key$.

Left subtree keys $< x.key <$ Right subtree keys

Convention

These slides use distinct keys. Duplicate insertions leave the tree unchanged.

Example of a Binary Search Tree



$$\{20, 30, 40\} < 50 < \{60, 70, 80\}.$$

The ordering property holds recursively at every node.

The Property Applies to Entire Subtrees

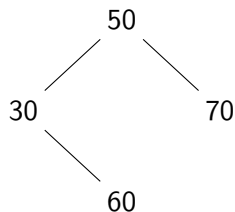
For a node with key 50:

- Every key in its left subtree must be below 50.
- Every key in its right subtree must be above 50.
- Checking only its immediate children is insufficient.

Recursive Structure

Both the left and right subtrees must themselves be BSTs, and they must satisfy the bounds imposed by their ancestors.

An Invalid BST



- $60 > 30$, so the local relation at 30 is valid.
- But 60 lies in the left subtree of 50.
- Therefore it must also satisfy $60 < 50$.

Conclusion

This tree is not a BST.

Binary Tree and Binary Search Tree

Property	Binary Tree	Binary Search Tree
Children per node	At most two	At most two
Key ordering	No required ordering	Left keys smaller, right keys larger
Inorder traversal	Not necessarily sorted	Sorted
Search strategy	May inspect all nodes	Follows one root-to-node path
Worst-case search	$O(n)$	$O(n)$ if highly unbalanced

Terminology and Conventions

- n : number of nodes.
- h : tree height, measured in edges.
- Root depth: 0.
- Leaf height: 0.
- Empty-tree height: -1 by convention.
- A path can contain at most $h + 1$ nodes.

Operation Cost

Search, insertion and deletion take $O(h + 1)$ time. This is commonly written as $O(h)$ for nontrivial trees.

Searching in a BST

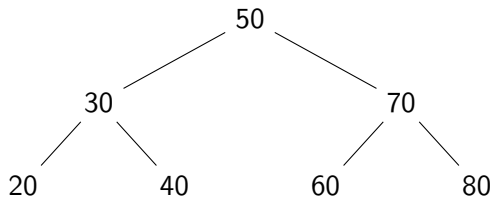
To search for key k at node x :

- ① If x is NIL, the key is absent.
- ② If $k = x.key$, the key is found.
- ③ If $k < x.key$, continue in the left subtree.
- ④ If $k > x.key$, continue in the right subtree.

Why One Subtree is Enough

The BST property rules out the other subtree.

Search Example: Find 60



$60 > 50,$ $60 < 70,$ $60 = 60.$

Visited nodes: 50, 70, 60

Unsuccessful Search: Find 65

Using the same BST:

$50 \longrightarrow 70 \longrightarrow 60.$

At node 60:

$65 > 60.$

Continue to its right child, which is NIL.

Key 65 is absent.

Insertion Position

The same search identifies where a new node with key 65 would be inserted: as the right child of 60.

Iterative Search Algorithm

```
BST-SEARCH(root, key)
    current = root

    while current != NIL
        if key == current.key
            return current

        if key < current.key
            current = current.left
        else
            current = current.right

    return NIL
```

Time = $O(h + 1)$, Auxiliary space = $O(1)$

Insertion into a BST

To insert a distinct key:

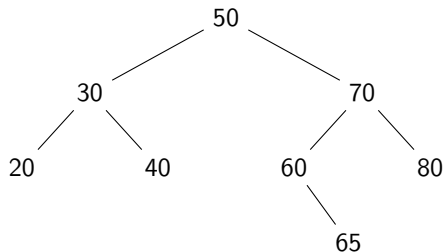
- ① Begin at the root.
- ② Move left or right using comparisons.
- ③ Stop when the required child pointer is NIL.
- ④ Attach the new node at that position.

Important

Ordinary BST insertion adds the new node as a leaf. It does not automatically rebalance the tree.

$$\text{Insertion time} = O(h + 1)$$

Insertion Example: Insert 65



$65 > 50$, $65 < 70$, $65 > 60$.

Recursive Insertion Algorithm

```
BST-INSERT(root, key)
    if root == NIL
        return a new node containing key

    if key < root.key
        root.left = BST-INSERT(root.left, key)

    else if key > root.key
        root.right = BST-INSERT(root.right, key)

    return root
```

Calling Convention

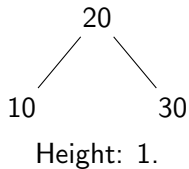
Assign the returned root:

root = BST-INSERT(*root*, *key*).

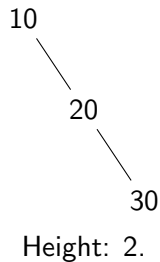
Insertion Order Determines Shape

The same keys can produce different trees.

Insert: 20, 10, 30.



Insert: 10, 20, 30.



Minimum and Maximum

Minimum

Start at the root and repeatedly follow left pointers.

The leftmost node contains the minimum.

Maximum

Start at the root and repeatedly follow right pointers.

The rightmost node contains the maximum.

$$\text{Time} = O(h + 1)$$

For the standard example:

$$\text{min} = 20, \quad \text{max} = 80.$$

Minimum and Maximum Algorithms

```
BST-MINIMUM(root)
    if root == NIL
        return NIL
    while root.left != NIL
        root = root.left
    return root
```

```
BST-MAXIMUM(root)
    if root == NIL
        return NIL
    while root.right != NIL
        root = root.right
    return root
```

Inorder Traversal Produces Sorted Keys

Inorder visits:

Left subtree, Root, Right subtree.

For a BST:

Left keys $<$ Root key $<$ Right keys.

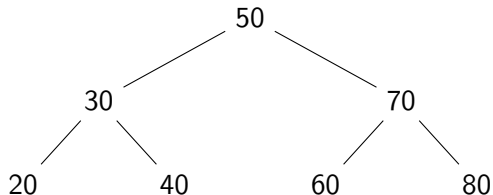
Therefore:

Inorder traversal gives increasing order.

For the standard example:

20, 30, 40, 50, 60, 70, 80.

All Traversals of the Example BST



Traversal	Sequence
Preorder	50, 30, 20, 40, 70, 60, 80
Inorder	20, 30, 40, 50, 60, 70, 80
Postorder	20, 40, 30, 60, 80, 70, 50
Level order	50, 30, 70, 20, 40, 60, 80

Successor and Predecessor

Inorder Successor

The successor of a node is the node with the smallest key strictly greater than its key.

Inorder Predecessor

The predecessor of a node is the node with the largest key strictly smaller than its key.

For the standard example:

$$\begin{aligned}\text{successor}(50) &= 60, \\ \text{predecessor}(50) &= 40.\end{aligned}$$

Finding the Successor

Case 1: A Right Subtree Exists

The successor is the minimum node in the right subtree.

Case 2: No Right Subtree Exists

Move upward until reaching the first ancestor for which the current path comes from its left subtree. That ancestor is the successor.

If there is no such ancestor, the node is the maximum and has no successor.

$$\text{Time} = O(h + 1)$$

Successor Examples

Using the standard BST:

- Successor of 50: minimum in its right subtree is 60.
- Successor of 40: move upward through 30 to reach 50.
- Successor of 60: its parent 70 is the answer.
- Successor of 80: none, because 80 is the maximum.

Predecessor

Use the symmetric rules: maximum in the left subtree, or the first suitable ancestor reached from its right subtree.

Successor Without Parent Pointers

```
SUCCESSOR(root, key)
    answer = NIL

    while root != NIL
        if key < root.key
            answer = root
            root = root.left
        else
            root = root.right

    return answer
```

Meaning

Returns the smallest stored key greater than key. This also works when the queried key is absent.

Deletion from a BST

Deletion has three structural cases:

- 1 The node is a leaf.
- 2 The node has exactly one child.
- 3 The node has two children.

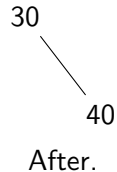
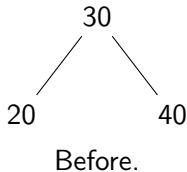
Requirement

The remaining structure must preserve the BST property.

$$\text{Deletion time} = O(h + 1)$$

Deletion Case 1: Leaf Node

Delete 20.

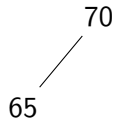
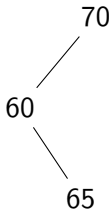


Action

Set the parent's corresponding child pointer to NIL, then release the removed node.

Deletion Case 2: One Child

Delete 60, whose only child is 65.



Action

Connect the parent directly to the deleted node's child. If deleting the root, the child becomes the new root.

Deletion Case 3: Two Children

A node with two children can be replaced using:

- Its inorder successor, or
- Its inorder predecessor.

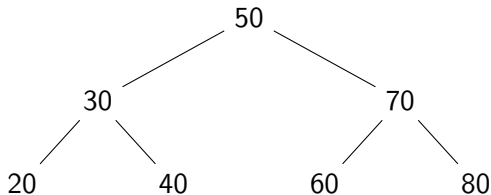
Using the successor:

- 1 Find the minimum in the right subtree.
- 2 Copy its key and associated data into the target node.
- 3 Delete that successor from the right subtree.

Why This Reduces the Problem

The successor has no left child. Its removal therefore uses the leaf or one-child case.

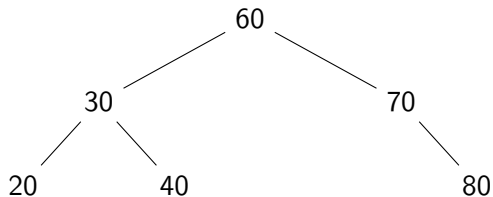
Two-Child Deletion: Delete 50



The successor of 50 is 60.

Copy 60 into the root, then remove the original node 60.

Two-Child Deletion: Final Tree



Inorder:

20, 30, 40, 60, 70, 80

.

A Successor May Have a Right Child

Suppose the right subtree of 50 contains:

70 with left child 60,

and node 60 has right child 65.

When deleting 50:

- 1 Copy key 60 into the target node.
- 2 Remove the original node 60.
- 3 Reconnect its right child 65 as the left child of 70.

Important

The successor has no left child, but it may have a right child. That child must remain connected.

Deletion Algorithm: Search and Simple Cases

```
BST-DELETE(root, key)
    if root == NIL
        return NIL

    if key < root.key
        root.left = BST-DELETE(root.left, key)

    else if key > root.key
        root.right = BST-DELETE(root.right, key)

    else
        if root.left == NIL
            save root.right, release root
            return the saved child

        if root.right == NIL
            save root.left, release root
```

Deletion Algorithm: Two-Child Case

Continue inside the matching-key case:

```
    successor = BST-MINIMUM(root.right)

    copy successor's key and associated data
        into root

    root.right =
        BST-DELETE(root.right, successor.key)

return root
```

Calling Convention

Always retain the returned root:

$$root = \text{BST-DELETE}(root, key).$$

Why Successor Replacement is Correct

Let the deleted key be k and its successor be s .

- Every left-subtree key is smaller than k .
- Since $k < s$, every left-subtree key is smaller than s .
- s is the smallest key in the right subtree.
- After removing the original s , every remaining right-subtree key is larger than s .

Therefore:

Left keys $< s <$ Remaining right keys.

Node Identity During Deletion

The previous algorithm copies an item's key and data.

- The target node object remains in memory.
- The successor node object is released.
- External pointers to nodes may therefore need care.

Alternative

A transplant-based implementation moves node objects by updating pointers instead of copying item contents.

The following C implementation stores only integer keys and uses the copying approach.

C Implementation: Node and Allocation

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int key;
    struct Node *left;
    struct Node *right;
} Node;

Node *newNode(int key)
{
    Node *p = malloc(sizeof *p);
    if (p == NULL) {
        fprintf(stderr, "Allocation failed\n");
        exit(EXIT_FAILURE);
    }
    p->key = key;
    p->left = p->right = NULL;
```

C Implementation: Insertion

```
Node *insert(Node *root, int key)
{
    if (root == NULL)
        return newNode(key);

    if (key < root->key)
        root->left = insert(root->left, key);

    else if (key > root->key)
        root->right = insert(root->right, key);

    return root;
}
```

Duplicate Policy

If the key is already present, return the tree unchanged.

C Implementation: Search

```
Node *search(Node *root, int key)
{
    while (root != NULL) {
        if (key == root->key)
            return root;

        if (key < root->key)
            root = root->left;
        else
            root = root->right;
    }

    return NULL;
}
```

Auxiliary space = $O(1)$

C Implementation: Minimum and Maximum

```
Node *minimum(Node *root)
{
    if (root == NULL)
        return NULL;
    while (root->left != NULL)
        root = root->left;
    return root;
}

Node *maximum(Node *root)
{
    if (root == NULL)
        return NULL;
    while (root->right != NULL)
        root = root->right;
    return root;
}
```

C Implementation: Delete, Part 1

```
Node *deleteKey(Node *root, int key)
{
    if (root == NULL)
        return NULL;

    if (key < root->key) {
        root->left = deleteKey(root->left, key);
    } else if (key > root->key) {
        root->right = deleteKey(root->right, key);
    } else {
        if (root->left == NULL) {
            Node *child = root->right;
            free(root);
            return child;
        }
        if (root->right == NULL) {
            Node *child = root->left;
            free(root);
            return child;
        }
    }
}
```

C Implementation: Delete, Part 2

Continue the function from the preceding slide:

```
Node *s = minimum(root->right);  
root->key = s->key;  
  
root->right =  
    deleteKey(root->right, s->key);  
}  
  
return root;  
}
```

Deletion of an Absent Key

The search reaches NULL and returns without changing the stored keys.

C Implementation: Inorder and Cleanup

```
void inorder(const Node *root)
{
    if (root == NULL)
        return;
    inorder(root->left);
    printf("%d ", root->key);
    inorder(root->right);
}

void freeTree(Node *root)
{
    if (root == NULL)
        return;
    freeTree(root->left);
    freeTree(root->right);
    free(root);
}
```

Cleanup uses postorder so children are released before parents

C Implementation: Example Driver

```
1 int main(void)
2 {
3     int keys[] = {50, 30, 70, 20, 40, 60, 80};
4     size_t n = sizeof keys / sizeof keys[0];
5     Node *root = NULL;
6
7     for (size_t i = 0; i < n; ++i)
8         root = insert(root, keys[i]);
9
10    printf("Inorder: ");
11    inorder(root);
12
13    printf("\nSearch 60: %s\n",
14          search(root, 60) ? "Found" : "Not found");
15
16    root = deleteKey(root, 50);
17    printf("After deleting 50: ");
18    inorder(root);
19    printf("\n");
20 }
```

C Program: Expected Output

```
Inorder: 20 30 40 50 60 70 80  
Search 60: Found  
After deleting 50: 20 30 40 60 70 80
```

Combining the Program

Place the C snippets in their presented order. Join the two deletion snippets into one function.

Recursive Depth

Recursive insertion, deletion, traversal and cleanup can use linear call-stack space for a skewed tree.

Height Determines Performance

For a non-empty BST containing n nodes:

$$\lceil \log_2(n+1) \rceil - 1 \leq h \leq n - 1.$$

- A balanced shape has $h = O(\log n)$.
- A completely skewed shape has $h = n - 1$.
- Ordinary BST insertion does not enforce balance.

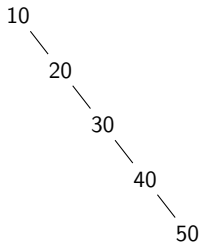
Consequence

Search, insertion and deletion are logarithmic only when the height is logarithmic.

Worst Case: Sorted Insertion

Insert:

10, 20, 30, 40, 50.



- Every insertion moves right.
- Height becomes $n - 1$.
- Searching for 50 visits all five nodes.
- The tree behaves like a linked list.

Operation Complexity

Operation	Height $O(\log n)$	Worst-Case Shape
Search	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(n)$
Minimum / maximum	$O(\log n)$	$O(n)$
Successor / predecessor	$O(\log n)$	$O(n)$
Full traversal	$\Theta(n)$	$\Theta(n)$

Traversal

Visiting every node takes linear time regardless of tree shape.

Building a BST by Repeated Insertion

For sorted input, the insertion path lengths grow as:

$$0, 1, 2, \dots, n-1.$$

The total is:

$$\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}.$$

Therefore:

$$\text{Worst-case construction time} = \Theta(n^2).$$

When Height Remains Logarithmic

Repeated insertion takes $O(n \log n)$ total time. A self-balancing BST maintains this guarantee.

Balanced BST from a Sorted Array

Given a sorted array:

- 1 Choose its middle element as the root.
- 2 Build the left subtree from the left half.
- 3 Build the right subtree from the right half.

For:

[20, 30, 40, 50, 60, 70, 80],

choose 50 as root, then 30 and 70 as subtree roots.

Construction time = $\Theta(n)$

Resulting height = $O(\log n)$

Validating a BST

Checking only parent-child pairs is insufficient.

Each node must satisfy an allowed interval:

$$low < key < high.$$

- Left recursion changes the upper bound to the current key.
- Right recursion changes the lower bound to the current key.
- Ancestor restrictions are preserved throughout recursion.

$$\text{Validation time} = O(n)$$

BST Validation Algorithm

```
VALIDATE(root, low, high)
    if root == NIL
        return TRUE

    if root.key <= low or root.key >= high
        return FALSE

    return VALIDATE(root.left, low, root.key)
           and VALIDATE(root.right, root.key, high)
```

Initial conceptual bounds:

$$(-\infty, +\infty).$$

In code, use optional bounds or a safely wider type. Do not compute $\text{key}-1$ or $\text{key}+1$, which may overflow.

Duplicate Keys

Several policies are possible:

- Reject duplicate insertions.
- Store a frequency count in each node.
- Store a collection of records for each distinct key.
- Use a consistent side rule with a modified invariant.

Policy Used Here

One node per distinct key. Inorder traversal is strictly increasing.

All operations and validation rules must follow the same policy.

BST and Binary Heap

Property	BST	Binary Min-Heap
Ordering	Left subtree $<$ node $<$ right subtree	Parent $\leq$ children
Shape	Arbitrary unless balanced	Complete binary tree
Root	Not necessarily minimum	Minimum key
Inorder	Sorted	Not necessarily sorted
Arbitrary search	$O(h + 1)$	$O(n)$ worst case

Applications of BSTs

- Maintaining a dynamic ordered set.
- Implementing ordered dictionaries.
- Finding predecessor and successor keys.
- Reporting keys in a specified range.
- Producing sorted output by inorder traversal.
- Supporting rank and selection with subtree-size augmentation.

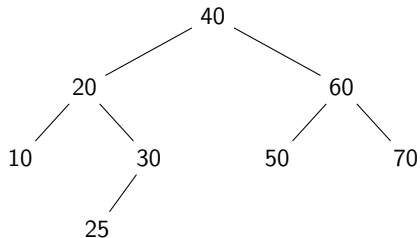
Balanced Variants

AVL trees and red-black trees maintain logarithmic height under insertions and deletions.

Worked Problem 1: Construct a BST

Insert in this order:

40, 20, 60, 10, 30, 50, 70, 25.



Worked Problem 1: Traversals and Height

For the constructed tree:

Inorder

10, 20, 25, 30, 40, 50, 60, 70

Preorder

40, 20, 10, 30, 25, 60, 50, 70

Postorder

10, 25, 30, 20, 50, 70, 60, 40

$h = 3$ edges

Worked Problem 2: Search Cost

Using the preceding tree, search for 25.

Visited nodes:

40, 20, 30, 25.

Decisions:

$25 < 40$, $25 > 20$, $25 < 30$, $25 = 25$.

Nodes examined = 4

Counting Convention

This counts one key comparison per visited node in an abstract three-way comparison model. Individual C relational tests may have a different count.

Worked Problem 3: Delete a Two-Child Node

In the worked tree, delete 20.

Its right subtree is rooted at 30 and contains 25.

$$\text{successor}(20) = 25.$$

- 1 Replace key 20 with 25.
- 2 Delete the original leaf node containing 25.

The resulting inorder traversal is:

10, 25, 30, 40, 50, 60, 70.

Worked Problem 4: Possible Heights

A BST contains 15 nodes.

Minimum possible height:

$$h_{\min} = \lceil \log_2(15 + 1) \rceil - 1 = 4 - 1 = \boxed{3}.$$

Maximum possible height:

$$h_{\max} = 15 - 1 = \boxed{14}.$$

Interpretation

The same number of keys can produce very different operation costs depending on tree shape.

Common Mistakes

- Checking only immediate children when validating.
- Assuming every BST is balanced.
- Claiming all BST operations always take $O(\log n)$.
- Losing a child subtree during deletion.
- Forgetting to assign the returned root.
- Forgetting that a successor may have a right child.
- Mixing duplicate-key conventions.

Practice Questions

Insert:

45, 25, 65, 15, 35, 55, 75, 30.

- 1 Draw the resulting BST.
- 2 Write its inorder and preorder traversals.
- 3 Find its height.
- 4 Find the successor and predecessor of 35.
- 5 Delete 25 using its inorder successor.
- 6 Give the search path for absent key 32 in the original tree.

Practice Answers: Traversals

Inorder

15, 25, 30, 35, 45, 55, 65, 75

Preorder

45, 25, 15, 35, 30, 65, 55, 75

- Height: edges.
- Successor of 35: .
- Predecessor of 35: .

Practice Answers: Deletion and Search

Delete 25

Its successor is 30.

Replace key 25 with 30, then remove the original node 30.

New inorder:

15, 30, 35, 45, 55, 65, 75.

Search for 32 in the Original Tree

45 $\longrightarrow$ 25 $\longrightarrow$ 35 $\longrightarrow$ 30 $\longrightarrow$ NIL.

At 30, search continues right and fails.

Quick Revision

Concept	Key Point
BST property	Left keys < node < right keys
Search	Follow one comparison-guided path
Insertion	Attach at an available leaf position
Minimum	Leftmost node
Maximum	Rightmost node
Inorder traversal	Increasing order
Successor	Smallest strictly larger key
Two-child deletion	Use successor or predecessor
Operation time	Depends on height
Worst-case height	$n - 1$

Summary

- BST ordering applies to entire subtrees.
- Search and insertion follow one path.
- Deletion must reconnect the remaining subtrees.
- Inorder traversal produces sorted keys.
- Ordinary BSTs do not guarantee balance.
- Balanced variants maintain logarithmic height.

Search, insert and delete: $O(h + 1)$

Balanced: $O(\log n)$ Worst-case shape: $O(n)$

Reference

Textbook

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein.

Introduction to Algorithms, 3rd edition, MIT Press, 2009.

- Chapter 12: Binary Search Trees.
- Section 12.1: What is a binary search tree?
- Section 12.2: Querying a binary search tree.
- Section 12.3: Insertion and deletion.

Thank You